МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт искусственного интеллекта

Кафедра проблем управления

ПРАКТИЧЕСКАЯ РАБОТА №5

по дисциплине **Методы искусственного интеллекта**

Тема практической работы: «Кластеризация многомерных данных на основе метода К-средних»

Студент группы: КРБО-03-23 Зенина А. А. _____

Грачев А. В. _____

Преподаватель: доцент Золотарёв А.А. _____

Работа представлена к защите: «___» _____ 2026 г.

Москва 2026

1. Цель работы

Изучение особенностей кластеризации многомерных данных на основе метода K-средних.

2. Задача работы

1. Запустить среду разработки PyCharm Community Edition (или Visual Studio Code).
2. Создать проект с названием «pr5» в директории дисциплины.
3. Реализовать программную модель метода K-средних на языке Python.
4. Выполнить кластеризацию набора двумерных точек данных с тремя кластерами.
5. Визуализировать процесс кластеризации на 5 итерациях.
6. Выполнить дополнительные задания:

- Реализовать процедуру автоматического определения наилучшего числа кластеров методом локтя.
- Исследовать эффективность алгоритма кластеризации для линейно неразделимых наборов точек (окружности, полумесяцы).

3. Теоретические сведения

Мобильные роботы, оснащаемые сканирующим лидаром и RGBD-камерой, в процессе своего перемещения получают двухмерные и трехмерные облака точек, которые описывают объекты внешней среды.

Многомерными облаками точек могут быть также представлены статистические данные о событиях, регистрируемых в различного рода инфокоммуникационных и вычислительных системах, например информация о возрасте, регионе, платежеспособности покупателей интернет-магазина.

Важной задачей при этом является разделение полученного облака точек на фрагменты, относящиеся к отдельным классам объектов или явлений. Эффективным подходом к решению этой проблемы является метод кластеризации K-средних.

Основная идея метода заключается в формировании случайной гипотезы о расположении кластеров вблизи точек данных и последующем итеративном улучшении этой гипотезы — через перемещение соответствующих центров кластеров к наиболее подходящим позициям, а также за счет коррекции переменных принадлежности точек к ближайшим кластерам.

Алгоритм K-средних состоит из следующих шагов:

Шаг 1. Инициализация K выбранных кластеров c_1, …, c_K в случайных точках многомерного пространства.

Шаг 2. Обновление меток принадлежности каждой точки данных pi к ближайшему кластеру cj. Степень близости оценивается как евклидово расстояние:

$d(p, c) = \sqrt{((xp - xc)^2 + (yp - yc)^2)}$

Шаг 3. Расчет геометрических центров $c_1^*, \ldots, c_K^*$ подмножеств точек, отнесенных к различным кластерам.

Шаг 4. Перемещение центров кластеров $c_1, \ldots, c_K$ в позиции $c_1^*, \ldots, c_K^*$.

Шаг 5. Если величина перемещения мала, то кластеризация считается завершенной, иначе осуществляется переход к Шагу 2.

Лабораторная установка: персональный компьютер с установленной средой разработки PyCharm Community Edition и интерпретатором Python 3.x с библиотеками PIL (Pillow), NumPy, matplotlib и scikit-learn.

4. Расчетно-графическая часть

4.1. Исходные данные

Для выполнения базового задания использовался набор из 11 точек на двумерной плоскости со следующими координатами:

```
pts = [
    [100, 70], [120, 80], [90, 110], [110, 130], [130, 90],
    [80, 140], [90, 120], [150, 125], [145, 45], [160, 130],
    [120, 100]
]
```

Начальные центры кластеров (K = 3) заданы в точках:

```
centers = [
    [90, 80, 0],
    [150, 80, 1],
    [60, 120, 2]
]
```

4.2. Результаты базового задания

В ходе выполнения программы было выполнено 5 итераций алгоритма K-средних. На рисунках 1-5 представлена визуализация процесса кластеризации на каждой итерации. Точки данных обозначены кругами, окрашенными в соответствии с принадлежностью к кластеру (черный, красный, зеленый). Центры кластеров обозначены квадратами соответствующих цветов.

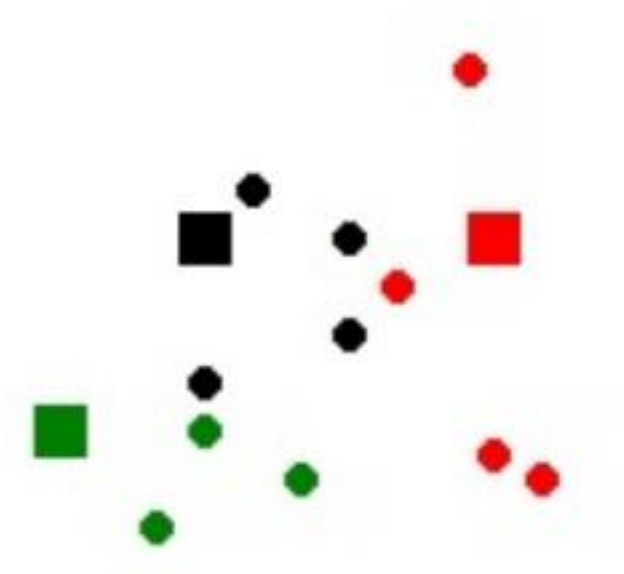

Рисунок 1 — Исходное расположение точек и начальные центры кластеров (итерация 0)

Рисунок 2 — Результат после 1-й итерации алгоритма K-средних

Рисунок 3 — Результат после 2-й итерации алгоритма K-средних

Рисунок 4 — Результат после 3-й итерации алгоритма K-средних

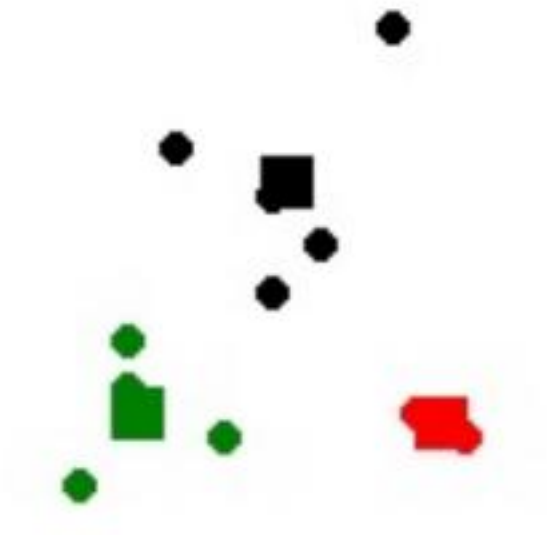

Рисунок 5 — Результат после 4-й итерации алгоритма K-средних (финальная кластеризация)

Анализ полученных изображений показывает, что уже после 3-й итерации центры кластеров стабилизировались, и дальнейшие итерации не приводят к изменению раскраски точек данных. Это свидетельствует о сходимости алгоритма.

4.3. Дополнительное задание 1: Метод локтя

Для автоматического определения оптимального числа кластеров был реализован метод локтя (elbow method). Суть метода заключается в вычислении суммы квадратов расстояний от точек до центров их кластеров (инерции) для различного числа кластеров k. Оптимальное значение k выбирается в точке, где график инерции образует «локоть» — резкое замедление скорости убывания функции.

Были рассчитаны значения инерции для k от 1 до 8:

Значения инерции:

- k=1: 15872.73
- k=2: 9727.50
- k=3: 7229.17
- k=4: 3283.33
- k=5: 2379.17

- k=6: 2400.00
- k=7: 629.17
- k=8: 212.50

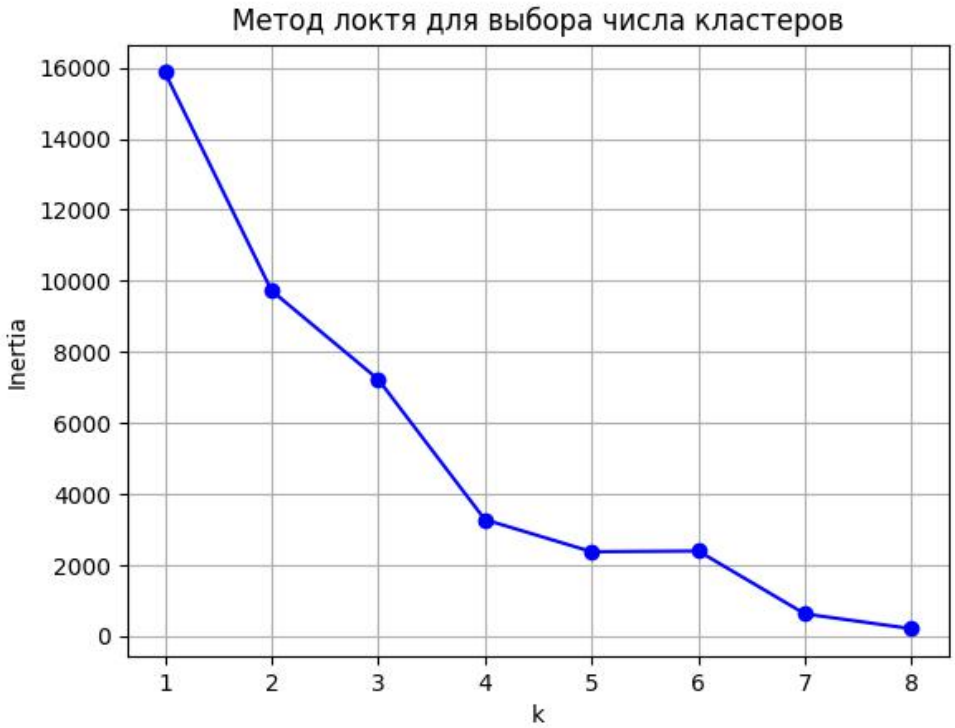

Рисунок 6 — График метода локтя для определения оптимального числа кластеров

Анализ графика показывает, что наиболее выраженный «локоть» наблюдается при k=5, что соответствует исходному предположению о количестве кластеров в данных.

4.4. Дополнительное задание 2: Линейно неразделимые данные

Для исследования эффективности алгоритма K-средних на линейно неразделимых данных были сгенерированы два набора синтетических данных:

- Концентрические окружности (circles): 300 точек, фактор масштаба 0.5, шум 0.05.
- Полумесяцы (moons): 300 точек, шум 0.05.

Для оценки качества кластеризации использовались метрики:

- ARI (Adjusted Rand Index) — скорректированный индекс Рэнда, измеряет сходство между истинными метками и результатами кластеризации. Значение 1.0 означает идеальное совпадение, 0 — случайное.
- NMI (Normalized Mutual Information) — нормализованная взаимная информация, также измеряет согласованность разбиений.

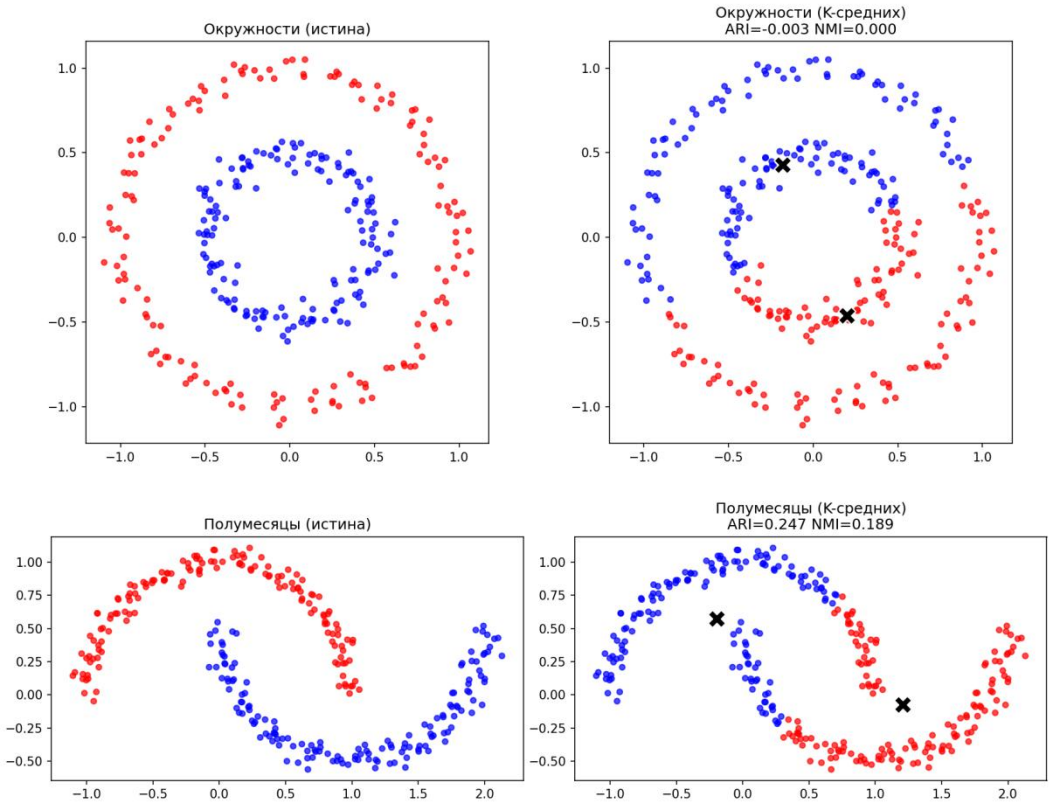

Рисунок 7 — Сравнение истинных меток и результатов кластеризации K-средних для линейно неразделимых данных

Результаты кластеризации:

- Окружности: ARI = -0.003, NMI = 0.000

- Полумесяцы: ARI = 0.247, NMI = 0.189

Полученные результаты демонстрируют, что стандартный алгоритм K-средних, основанный на евклидовом расстоянии, не способен корректно разделить линейно неразделимые данные. Для окружностей качество кластеризации практически равно случайному (ARI ≈ 0). Для полумесяцев результаты несколько лучше, но все еще далеки от идеальных. Это объясняется тем, что K-средних ищет компактные сферические кластеры и не может выделить кластеры сложной формы. Для таких данных требуются более продвинутые методы, такие как спектральная кластеризация или DBSCAN.

5. Выводы по работе

В ходе выполнения лабораторной работы были достигнуты следующие результаты:

1. Изучен алгоритм кластеризации K-средних, его основные шаги и принципы работы.

2. Реализована программная модель метода K-средних на языке Python с визуализацией процесса кластеризации.

3. Экспериментально подтверждена сходимость алгоритма — центры кластеров стабилизировались после 2-3 итераций.

4. Реализован метод локтя для автоматического определения оптимального числа кластеров. Для исследуемого набора данных оптимальным оказалось значение k=3.

5. Проведено исследование эффективности K-средних на линейно неразделимых данных. Установлено, что алгоритм не справляется с кластеризацией концентрических окружностей (ARI ≈ 0) и показывает низкое качество на данных типа «полумесяцы» (ARI ≈ 0.25).

Цель работы достигнута в полном объеме. Полученные практические навыки могут быть применены для решения задач сегментации облаков точек в робототехнических системах и анализа многомерных данных.

ПРИЛОЖЕНИЕ А

В приложении представлен исходный код программы на языке Python, реализующей алгоритм K-средних, метод локтя и тестирование на линейно неразделимых данных.

```
import math
import random
from dataclasses import dataclass
from typing import List, Tuple, Optional
from PIL import Image, ImageDraw
import matplotlib.pyplot as plt
from sklearn.metrics import adjusted_rand_score, normalized_mutual_info_score
from sklearn.datasets import make_moons, make_circles


@dataclass
class Point:
    """Класс для представления точки данных"""
    x: float
    y: float
    label: int = -1

    def coords(self) -> Tuple[float, float]:
        return (self.x, self.y)


def dist(p1: Point, p2: Point) -> float:
    """Евклидово расстояние между двумя точками"""
    dx = p1.x - p2.x
    dy = p1.y - p2.y
    return math.sqrt(dx*dx + dy*dy)


def find_nearest_center(point: Point, centers: List[Point]) -> int:
    """Возвращает индекс ближайшего центра кластера"""
    min_d = float('inf')
    best = 0
    for i, c in enumerate(centers):
        d = dist(point, c)
        if d < min_d:
            min_d = d
            best = i
    return best


def assign_points(pts: List[Point], centers: List[Point]) -> None:
    """Присваивает каждой точке метку ближайшего кластера"""
    for p in pts:
        p.label = find_nearest_center(p, centers)


def compute_center_of_mass(pts: List[Point], cluster_idx: int) ->
Optional[Point]:
    """Вычисляет центр масс для всех точек с заданной меткой"""
    xs = [p.x for p in pts if p.label == cluster_idx]
    ys = [p.y for p in pts if p.label == cluster_idx]
    if not xs:
        return None
    return Point(sum(xs)/len(xs), sum(ys)/len(ys), cluster_idx)
```

```
def shift_centers(pts: List[Point], centers: List[Point]) -> List[Point]:
    """Перемещает центры кластеров в центры масс соответствующих групп"""
    new_centers = []
    for idx, c in enumerate(centers):
        cm = compute_center_of_mass(pts, idx)
        if cm is None:
            new_centers.append(c)
        else:
            new_centers.append(cm)
    return new_centers


def kmeans(pts: List[Point], centers: List[Point], iterations: int = 5) ->
List[Point]:
    """Выполняет заданное количество итераций К-средних"""
    assign_points(pts, centers)
    for _ in range(iterations):
        centers = shift_centers(pts, centers)
        assign_points(pts, centers)
    return centers


def compute_inertia(pts: List[Point], centers: List[Point]) -> float:
    """Сумма квадратов расстояний от точек до их центров"""
    inertia = 0.0
    for p in pts:
        c = centers[p.label]
        inertia += dist(p, c) ** 2
    return inertia


def elbow_method(pts: List[Point], max_k: int = 8, iterations: int = 5) ->
List[float]:
    """Метод локтя для определения оптимального числа кластеров"""
    inertias = []
    for k in range(1, max_k + 1):
        indices = random.sample(range(len(pts)), k)
        centers = [Point(pts[i].x, pts[i].y, idx) for idx, i in
enumerate(indices)]
        pts_copy = [Point(p.x, p.y, p.label) for p in pts]
        assign_points(pts_copy, centers)
        for _ in range(iterations):
            centers = shift_centers(pts_copy, centers)
            assign_points(pts_copy, centers)
        inertia = compute_inertia(pts_copy, centers)
        inertias.append(inertia)
    return inertias


# Основная программа
if __name__ == "__main__":
    # Базовое задание
    pts = [
        Point(100, 70), Point(120, 80), Point(90, 110),
        Point(110, 130), Point(130, 90), Point(80, 140),
        Point(90, 120), Point(150, 125), Point(145, 45),
        Point(160, 130), Point(120, 100)
    ]
```

```
centers = [
    Point(90, 80, 0),
    Point(150, 80, 1),
    Point(60, 120, 2)
]

# Выполнение кластеризации и сохранение изображений
for i in range(5):
    if i > 0:
        centers = shift_centers(pts, centers)
    assign_points(pts, centers)

inertias = elbow_method(pts, max_k=8, iterations=5)
```